

Reflection on Teamwork

From the start of the assignment, we split work mostly by use case, with individuals writing both the code and tests for parts of the system. This was complimented by collaborative code reviews, where another team member would inspect and comment on the code and tests of another. As a result, code was written to fit the requirements of the tests and individuals generally had good working knowledge of their corner of the system. The code reviews did raise issues where people's code too closely fit their own coding style, to the detriment of others' understandings. Although such cases could be remedied, we realise that greater communication regarding programming conventions would have produced more consistent code. If restarting the project, we would make early decisions about high-level implementation and standardise coding styles.

We primarily communicated mainly through WhatsApp and MS Teams meetings and collated our work on GitHub. Our communication methods helped us conduct the aforementioned reviews. However, when unable to meet, we found textual communication limited for fairly complex discussion. GitHub facilitated the management of progress as well as our reviews. However, an infrequency of commits and some limited confusion about responsibilities made merges problematic. If we were to restart the coursework, our communication would be similar but more in-person to enable more dynamic communication. On the side of tools used for teamwork, we acknowledge that smaller, more frequent commits would have helped us keep track of each other's progress.

Reflection on Quality and Quality Attributes

We attempted to follow the design faithfully. In writing tests, we wanted to cover every possibility by inspecting branches of code and acting as users. In implementation, we believe to have made only marginal deviations from the given design, which lead to a fairly well-built solution. However, there were cases where code was rewritten because a small change from the design necessitated more which became unsustainable. We wrote a large number of different tests to simulate every situation that we could conceive, with descriptive messages to aid debugging. From this assignment, we learned the importance of keeping with the design, as deviations could snowball. We also discovered the difficulty of total test coverage, as it took many eyes to find gaps therein. If restarting, we would take time as a team to fully understand and talk through the design to resolve confusion or ambiguities immediately.

Regarding its use of personal and commercial data, security was an important consideration throughout construction. To protect the system, the scope of passwords was limited as much as possible through encapsulation. Furthermore, login required both usernames and passwords before errors were raised, so that neither could be

guessed individually. Since passwords came from the University, it was assumed that they were strong enough. We believe these efforts appreciated the significance of security and maintained it throughout the system. In building the project, we gained a deep understanding of the difficulty of limiting password scope and would have considered a simple form of encryption if restarting.

Reflection on Tools, Agile Approaches and Practices

As mentioned above, we made use of GitHub to centralise code, especially in service of our code reviews. The agile approach we used was Kanban, particularly its practices of visualisation of workflow and limitation of work-in-progress. We maintained a digital Kanban board using the Jira tool. It detailed tasks' journeys from plans to completed features. It was a priority of ours to minimise tasks 'in progress' in order to streamline work. As a result of Kanban, we found ourselves having specific work to focus on at any time, which we believed improved our productivity. However, there were occasional slips where tasks were started and then, for various reasons, attention was refocussed away from it. This created build-up in the active column of the board and divided focus suboptimally.

We chose to forgo an Extreme Programming (XP) approach because we did not have any equivalent of an on-site customer and because the system does not have a clear priority of features. Scrum was not employed as the roles of Product Owner and Scrum Master would leave too few people for development. More generally, Scrum's time management was too stringent for our needs. From our use of Kanban, we learned the effect of actively managing work in progress. Although we found success with our choices, if we were to start the coursework, we would expand our use of tools to aid our agile development. In particular Jira's bug tracking tool would have facilitated communication about issues in the code.

 **SEPPCoursework3** Private Watch 0

 main

 1 Branch

 0 Tags

 Add file

 Code

 **Mehedi-Alam** Finished EventPerformanceController ef6b847 · 19 hours ago 13 Commits

 CW3codeFiles	Finished EventPerformanceController	19 hours ago
 CodeReviewGroup39	Delete CodeReviewGroup39/CodeReviewGroup39.zip	3 days ago
 README.md	Initial commit	4 days ago

TO DO 4

Code: Register Entertainment Provider

☒ KAN-6

Code: View Performance

☒ KAN-9

Code: Edit Preferences

☒ KAN-11

Code: Review Performance

☒ KAN-14

+ Create

IN PROGRESS 3

Unit Tests: Booking

☒ KAN-3

Code: Search for Performances

☒ KAN-8

System Tests

☒ KAN-16

IN REVIEW 3

Unit Tests: MockSystemPayment

☒ KAN-2

Code: Log in

☒ KAN-4

Code: Log out

☒ KAN-5

DONE 6

Unit Tests: Performance

☒ KAN-1

Code: Create Event

☒ KAN-7

Code: Cancel Performance

☒ KAN-12

Code: Sponsor Performance

☒ KAN-15

Code: Cancel Booking

☒ KAN-13

Code: Book Performance

☒ KAN-10